

Guillaume Trüeb

Résumé DMA TE2

26-05-2026

Protocoles de proximité

USB

- Présent sur tous les mobiles récents, fiable pour gros volumes de données
- Smartphone peut être **périphérique** (ex: debug Android Studio) ou **hôte** (brancher un périphérique)

USB-C

- Connecteur réversible 24 pins : données + alimentation
- **Power Delivery** :
 - Base : 5V x 0,5A = 2,5W
 - Max : 48V x 5A = 240W
- Standards de débit :
 - USB 2.0 : 480 Mbps (2000)
 - USB 3.2 Gen 2 : 10 Gbps (2013)
 - USB 4 v2.0 : 80/120 Gbps (2022)
- **Thunderbolt** = version certifiée d'USB (TB3 → base d'USB4)
- Aucun smartphone n'embarque USB4/Thunderbolt en 2026
- ⚠ **Les câbles USB-C se ressemblent tous physiquement** mais peuvent avoir des capacités très différentes (USB 2.0 vs USB4, 60W vs 240W) → toujours vérifier les spécifications du câble

Android

- Périphériques supportés nativement : stockage, clavier, souris, Ethernet...
- SDK permet d'intégrer un **pilote USB custom** pour périphériques spécifiques

iOS

- Accessoires non natifs → **programme MFi** (certification Apple obligatoire)
- Coût 4\$ par accessoire vendu, NDA strict
- Lightning : débit USB 2.0 → iPhone 15 : transition vers USB 3.2 Gen 2
- Alternatives : port jack (supprimé en 2016), Ethernet nativement supporté

Codes-barres

1D

- Données encodées dans l'épaisseur/espacement des barres
- Formats : Code-128, EAN-13, UPC-A, Code-39...
- Lecture par **laser**, densité d'information faible → surtout pour identifiants

2D

- Formats : **QR Code**, Data Matrix, Aztec, PDF417...
- Lecture par **analyse photo**
- QR Code : correction d'erreurs intégrée (4 niveaux : L=7%, M=15%, Q=25%, H=30%)
- Capacité max : 7'089 chiffres / 4'296 alphanum / 2'953 ISO-8859-1
- Tailles : Version 1 (21x21) → Version 40 (177x177)

Types de contenu QR

- URL, vCard, événement, géolocalisation, Wi-Fi, email, téléphone...
- **Texte libre** : JSON, données custom (ex: certificat COVID en Base45)
- **GS1** : remplacera les EAN d'ici 2027 (identifiant + URI produit + métadonnées)
- **Dynamiques** : contenu modifiable, redirection selon OS, statistiques d'usage
 - ⚠ Confiance au fournisseur requise (disponibilité, vie privée)

Lecture sur Android

- **zxing** : librairie Java, licence Apache, mode maintenance
- **ML Kit** : librairie Google ML, modèles entraînés, Terms of Service Google

Exercice 4 — CameraX + MLKit

Problème avec les QR codes grandes versions (v25, v40) : résolution par défaut de CameraX = **640×480** → insuffisant. Règle : min 2 px par module QR, en pratique 4-8 px. Solution → forcer la résolution à **5 MP** :
// Exercice 4 – résolution CameraX pour grands QR codes
val resolutionSelector = ResolutionSelector.Builder()
 .setResolutionStrategy(ResolutionStrategy(
 Size(**2592**, **1920**), // SMP
 ResolutionStrategy.**FALLBACK_RULE_CLOSEST_HIGHER_THEN_LOWER**))
 .build()

```
val barcodesUseCase = ImageAnalysis.Builder()  
    .setResolutionSelector(resolutionSelector)  
    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST) //  
une image à la fois  
    .build().apply {  
        setAnalyzer(Executors.newSingleThreadExecutor()) { proxy ->  
processImageProxy(proxy) }  
    }
```

```
// Limiter aux formats utiles (meilleure performance)  
val barcodeClient = BarcodeScanning.getClient(  
BarcodeScannerOptions.Builder().setBarcodeFormats(Barcode.FORMAT_QR_CODE).build()  
)  
// Fermer le proxy après traitement !  
imageProxy.image?.close(); imageProxy.close()
```

Conclusion : QR v40 (177×177) utilisable techniquement mais pas adapté au grand public (difficile à scanner en conditions réelles).

NFC

- Classe particulière de **RFID**, lecture à max 3-4 cm
- Tag peut être **passif** (alimenté par champ EM du lecteur)
- Communication **bidirectionnelle**

Modes de fonctionnement

- **HCE** (émulation de carte) : terminal mobile agit comme carte sans contact (paiement)
- **Lecture/Écriture** : lire/écrire des tags passifs
- **Peer-to-peer** : échange direct entre 2 appareils NFC

Technologies & Tags

- NFC-A/B/F/V, MifareClassic, MifareUltralight
- Tags courants : NTAG213 (144b), NTAG216 (888b), SLIX (112b)

NDEF

- Format standardisé de messages lisibles sur la plupart des tags
- Types Well-Known : **URI**, **TEXT**, **SMARTPOSTER**
- Travailler au plus haut niveau d'abstraction recommandé

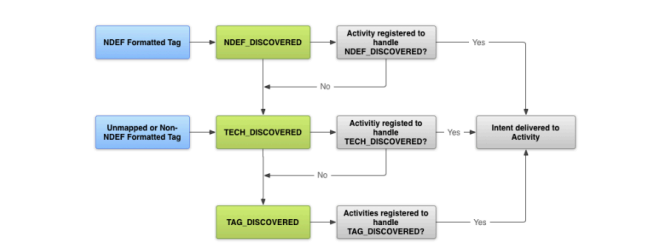
Android vs iOS

- **Android** : SDK très complet, tous niveaux d'abstraction, quasi tous modèles équipés
- **iOS** :
 - HW depuis 2014, API tierce depuis 2017 (lecture NDEF uniquement)
 - 2019 : écriture NDEF, 2022 : Commission EU accuse Apple d'abus de position dominante
 - iOS 18.1 : API étendue pour paiements/billetterie, pays et devs sélectionnés uniquement

Utilisation Android

```
// Permission requise dans le Manifest  
<uses-permission android:name="android.permission.NFC" />
```

```
// 3 niveaux d'abstraction (du plus spécifique au plus général)  
ACTION_NDEF_DISCOVERED // tag NDEF → niveau recommandé  
ACTION_TECH_DISCOVERED // technologie spécifique (NFC-A, Mifare, etc.)  
ACTION_TAG_DISCOVERED // n'importe quel tag NFC
```



Exercice 5 — ForegroundDispatch (lire quand l'activité est active)

```
// onResume → activer  
override fun onResume() {  
    super.onResume()  
    val intent = Intent(applicationContext, this::class.java).apply {  
        flags = Intent.FLAG_ACTIVITY_SINGLE_TOP // évite plusieurs instances  
        val pendingIntent = PendingIntent.getActivity(  
            applicationContext, 0, intent, PendingIntent.FLAG_MUTABLE) // FLAG_MUTABLE requis API 31+  
        val filters =  
arrayOf(IntentFilter(NfcAdapter.ACTION_NDEF_DISCOVERED))  
        val techLists = arrayOf(arrayOf("android.nfc.tech.Ndef"))  
        nfcAdapter.enableForegroundDispatch(this, pendingIntent, filters, techLists)  
    }
```

```
// onPause → désactiver  
override fun onPause() { super.onPause();  
nfcAdapter.disableForegroundDispatch(this) }
```

```
// Réception du tag via onNewIntent  
override fun onNewIntent(intent: Intent) {  
    super.onNewIntent(intent)  
    val rawMessages =  
intent.getParcelableArrayExtra(NfcAdapter.EXTRA_NDEF_MESSAGES)  
    rawMessages?.forEach { msg ->  
        (msg as NdefMessage).records.forEach { record -> /* parser */ }  
    }  
}
```

Format des enregistrements NDEF Well-Known (exercice 5)

RTD_TEXT	RTD_URI
Byte 0 : flags (bit7=encodage, bits5-0=longueur lang) Puis code langue (ASCII) Puis texte (UTF-8 ou UTF-16)	Byte 0 : préfixe URI 0x01=http://www. 0x02=https://www. 0x03=http:// 0x04=https:// Puis reste de l'URI (UTF-8)

```
// Décodage RTD_TEXT  
val flags = payload[0].toInt()  
val textEncoding = if ((flags and 0x80) == 0) Charsets.UTF_8 else  
Charsets.UTF_16  
val langLen = flags and 0x3F  
val lang = String(payload, 1, langLen, Charsets.US_ASCII)  
val text = String(payload, langLen + 1, payload.size - langLen - 1,  
textEncoding)
```

```
// Décodage RTD_URI → Android le fait automatiquement  
val uri = record.toUri()
```

Lecture depuis le Manifest (app inactive) : ajouter intent-filter sur l'activité → android:launchMode="singleTop" pour éviter plusieurs instances.

Depuis Android/iOS (2018) : le système lit **automatiquement** les tags NDEF au déverrouillage de l'écran et propose d'ouvrir l'URL ou l'app correspondante — sans qu'une app dédiée soit active. Une URL NDEF sur un tag est traitée comme un lien universel.

Bluetooth Classique

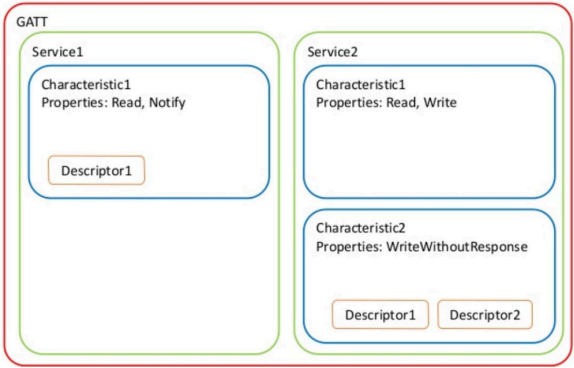
- Alternative sans-fil à USB, portée 10m, connexion bidirectionnelle chiffrée
- Peu adapté à la mobilité (consommation)
- Évolution : 1.0 (1999, 721 kbps) → 3.0 (2009) → 4.0 (2010, définit BLE)

Profiles principaux

- **A2DP** : streaming audio (casque, enceinte)
- **HFP/HSP** : kit mains-libres
- **HID/HOGP** : clavier, souris, manette
- **PAN** : partage de connexion
- **PBAP** : accès au répertoire téléphonique (combiné HFP pour voiture)
- **AVRCP** : télécommande multimédia (TV, audio)
- **MAP** : accès aux messages
- Profil custom → **MFi obligatoire sur iOS**, possible mais complexe sur Android
- iOS supporte nativement : HFP 1.7, PBAP 1.2, A2DP 1.3, AVRCP 1.6, PAN, HID, MAP 1.1, Braille

Bluetooth Low Energy (BLE)

- Technologie **indépendante** du BT classique (non compatible)
- Portée 5-100m, débit utile 100 kbps, fonctionne des **mois sur pile bouton**
- Topologie **client/serveur**



Phases de communication

- **GAP** (avant connexion) : diffusion nom, services disponibles, connexion possible ?
- **GATT** (connecté) : services, characteristics, descripteurs

Structure GATT

- **Service** = ensemble de Characteristics (ex: Heart Rate 0x180D, Battery 0x180F, Current Time 0x1805)
 - Services standards → UUID 16 bits ; propriétaires → UUID 128 bits
- **Characteristic** = variable (max **20 bytes** par défaut = 1 paquet, **512 bytes** max)
 - Opérations : **Lecture**, **Écriture** (avec/sans confirmation), **Notification** (sans ACK) / **Indication** (avec ACK)
 - Le Central doit s'**inscrire** pour recevoir des notifications
- **Descriptor** = métadonnées d'une Characteristic

Exemple : Current Time Service 0x1805 (format des bytes de la Characteristic 0x2A2B) :

Bytes	Contenu	Valeur ex.
0-1	Année (little-endian)	0xE6 0x07 = 2022
2	Mois (1-12)	0x03 = mars
3	Jour (1-31)	0x0F = 15
4	Heure (0-23)	0x0E = 14h

5	Minute (0-59)	0x1E = 30
6	Seconde (0-59)	0x00
7	Jour semaine (1=lundi)	0x02 = mardi
8	Fractions/256 s	0x00
9	Adjust reason	0x00

Sécurité / Appairage (Bonding)

- **Connecté** (non appairé) : échange en clair, tout appareil peut se connecter
- **Appairé/Bondé** : canal chiffré après échange de clés :
 - **Just Works** : automatique, vulnérable MITM
 - **Out of Band** : via NFC/Wi-Fi
 - **Passkey** : code PIN (écran + clavier)
 - **Numeric comparison** : 2 écrans, BLE 4.2+ uniquement

Évolutions BLE

- **5.0** (2016) : ×2 débit (2M PHY) ou ×4 portée (Coded PHY 200m), mesh networks
- **5.1** (2019) : calcul d'angle d'arrivée/départ (localisation en intérieur)
- **5.2** (2020) : profil audio BLE (LE Audio, remplace A2DP à terme)
- **5.3** (2021) : optimisations diverses
- **5.4** (2023) : chiffrement possible des annonces
- **6.0** (2024) : Channel Sounding (localisation précise)

Autres protocoles de proximité

- **Wi-Fi Direct** : connexion p2p Wi-Fi, idéal pour gros volumes
- **Google Nearby** : API p2p Android combinant BT + Wi-Fi + ultrasons
- **AirDrop** : Apple uniquement (macOS/iOS/iPadOS), BT + Wi-Fi, partage de fichiers

Synthèse & Bonnes Pratiques

Contraintes de Design Applicatif

Contrainte	Problème	Solution
Batterie	Émission = consommation	Regrouper requêtes, lazy poll, FCM
Latence	RTT élevé sur mobile	HTTP/3 QUIC, caching, look-ahead
Bande passante	Payload > capacité	Protobuf, DEFLATE, pagination
Connexion intermittente	Requête coupée = données perdues	Retry automatique, queue locale
Changement IP	Session TCP coupée	QUIC (connexion liée au token, pas IP)
Vie privée	Tracking FCM/NFC/BLE	Chiffrement E2E, permissions minimales
Multi-version	App ancienne + API nouvelle	Parseurs permissifs, versioning
Background	OS tue les apps	WorkManager, FCM, Foreground Service

Bonnes pratiques Android — Réseau & Performance

- **Ne jamais faire réseau/BDD sur le Main Thread** → ANR garanti → utiliser Dispatchers.io

- **Désabonner les listeners** en onPause()/onDestroy() (LocationManager, BLE, NFC ForegroundDispatch)
- **Limiter les permissions** demandées au strict nécessaire (foreground avant background)
- **Gérer les null** : lastLocation peut être null si aucune position connue
- **Fermer les proxies CameraX** : imageProxy.close() à la fin de chaque image, sinon freeze
- **Timeout cache iBeacon** : retirer balises non vues depuis > 5 s, pas en temps réel (trop instable)
- **Geofencing rayon mini** : 100-150 m minimum ; délai de notification : 5 min pour économiser batterie

Checklist de choix de protocole

Données volumineuses, bande passante critique	→ Protobuf + DEFLATE
Données simples, lisibilité nécessaire	→ JSON
Services inter-entreprises, contrats forts	→ SOAP (rarement sur mobile)
Requêtes flexibles, app riche	→ GraphQL
Notifications serveur → client	→ FCM + poll déclenché
Échange p2p local gros volume	→ Wi-Fi Direct
Déclenchement à courte portée	→ NFC (< 4cm)
Suivi de présence par salle	→ iBeacon BLE
Positionnement précis < 30cm	→ UWB (AirTag / Jetpack beta)

Capteurs

Généralités

- Couche d'abstraction matérielle (correction, calibration, uniformisation)
- **SensorManager** unifie l'accès, similaire à **LocationManager**
- Tous les capteurs ne sont pas présents sur tous les appareils → vérifier la disponibilité
- Évolution Galaxy : S(6) → S20(16 capteurs) — ajout fingerprint, heartrate, iris, barometer, hall effect

Capteurs simples (1 valeur)

Capteur	Unité	Notes
Luminosité	lx	face avant
Pression	hPa	atmosphérique → altitude
Proximité	cm	souvent binaire (on/off), désactive écran pendant appel
Humidité	%	humidité ambiante relative
Température	°C	ambiante

Capteurs de mouvement (3 valeurs, axes xyz)

- **Accéléromètre** [m·s⁻²] : accélération sur les 3 axes
 - Au repos, mesure la gravité → détection basculement/pivotement
 - Détecte marche/course/secousse/immobilité
 - Précision insuffisante pour pro
- **Magnétomètre** [µT] : intensité du champ magnétique
 - Position par rapport au pôle Nord magnétique
 - Très perturbé par masses métalliques et aimants
- **Gyroscope** [rad·s⁻¹] : vitesse de rotation
 - Plus rare (haut de gamme), consomme plus que l'accéléromètre
 - Plus rapide et précis pour calculer l'orientation
 - Utilisé pour AR/VR

Référence smartphone : dos = -z (vers centre Terre), haut = +y (nord magnétique), droite = +x (est)

Accès aux capteurs Android (SensorManager)

```
val sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
val accelerometer =
    sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
sensorManager.registerListener(listener, accelerometer,
    SensorManager.SENSOR_DELAY_NORMAL)
// [...]
sensorManager.unregisterListener(listener) // ▲ obligatoire en
onPause
```

Taux de rafraîchissement (du + lent au + rapide) :

- SENSOR_DELAY_NORMAL / _UI / _GAME / _FASTEST
- Depuis **Android 12** : accès > 200 Hz → permission particulière
- Résultats reçus sur le **UI Thread** → taux trop élevé = ralentit l’UI

SensorEventListener :

```
override fun onSensorChanged(event: SensorEvent) {
    when(event.sensor.type) {
        Sensor.TYPE_ACCELEROMETER -> event.values // float[3]
        Sensor.TYPE_LIGHT          -> event.values // float[1]
    }
}
override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
```

Exploitation des données brutes

- event.values = float[] dont la taille dépend du capteur
 - 1 valeur : pression, proximité, humidité…
 - 3 valeurs : accéléromètre, magnétomètre, gyroscope
 - 15 valeurs : TYPE_POSE_6DOF

Pool d’instances (SensorEvent) : pour éviter le GC sous pression (centaines d’évènements/s), le SensorManager **réutilise** les SensorEvent. **Ne pas conserver de référence** — traiter immédiatement ou copier les valeurs.

Helpers d’interprétation :

- SensorManager.getAltitude(p0, p) : altitude depuis pression au niveau de la mer + pression mesurée
- Variation rapide de pression → changement d’altitude
- Variation lente → tendance météo

Orientation 3D (exercice 6 — boussole 3D)

Matrice de rotation obtenue à partir de l’accéléromètre + magnétomètre : SensorManager.getRotationMatrix(float[] R, null, gravity, geomagnetic)
// R = matrice 4x4 sérialisée (16 floats)

Appliquée à un modèle 3D (via composition de rotations xyz) → orientation réelle du smartphone.

Rappel matrices 3D : identité, translation, scaling, rotation autour de X/Y/Z. La composition se fait par multiplication.

Capteurs composites (virtuels)

- Android fournit 15 capteurs virtuels qui fusionnent plusieurs capteurs physiques :
- Step counter / step detector** (podomètre) — accéléromètre principalement
 - Game rotation vector** — accéléromètre + magnétomètre + (gyroscope)
 - Linear acceleration** — accéléromètre + gyroscope (ou magnétomètre)

Analyse avancée — reconnaissance d’activité

4 groupes d’attributs :

- Environnemental** : température, bruit
- Mouvement** : accéléromètre, gyroscope, magnétomètre
- Localisation** : GPS
- Physiologique** : température corporelle, rythme cardiaque

- Un seul capteur ne suffit pas à classifier l’activité (ex: train vs voiture par vitesse moyenne) → fusion multi-capteurs nécessaire.
- ActivityRecognition** dans les Play Services (still, walking, running, biking, in_vehicle)
 - «Vos Trajets» Google Maps — redoutablement précis

Identification biométrique : les patterns de marche d’une personne peuvent l’identifier de manière unique.

Permissions & sécurité

- Avant Android 10 : accès libre aux capteurs
- Android 10 (2019)** : permission ACTIVITY_RECOGNITION pour podomètre et activity recognition
- Données brutes < 200 Hz toujours libres → études **TouchLogger** (>70% classification correcte des chiffres tapés via micro-rotations) et écoute de conversations via accéléromètre (vibrations du haut-parleur)

Usages détournés

- Capteur optique : color identifier, scan QR, OCR maths, mesure d’angles
- Pixel 4 (2019)** : radar Soli (échec commercial)
- iPhone 2020+** : LiDAR

Wearables

Définitions

- Wearable** ≠ **Handheld** (les deux se traduisent «portable»)
 - Wearable = vêtement/accessoire intelligent — informatique vestimentaire
 - Handheld = tenu en main (smartphone, tablette, scanner)
- Sous-catégorie de l’**Internet des Objets (IoT)**
- Conçu pour interaction sans bouton, fonctionne en permanence

3 catégories de wearables :

- Tracking, Measuring, Sensing** — capteurs corporels, UI limitée, données transmises pour analyse
- Réalité améliorée** — aide à la vie courante (navigation, rappels)
- Second écran (scaled down gadgets)** — extension du smartphone

Exemples

- Fitness/bien-être** : Oura Ring (sommeil), Garmin HRM-Pro (cardiaque), FINIS Smart Goggle (natation), WHOOP (multi-capteurs)
- Médical** : monitoring glucose continu, détection ulcères du pied (diabétiques), mesure ingestion d’eau, temp tech tattoos (activité musculaire)

Body Area Network (BAN)

- Réseau personnel autour du corps, **smartphone au centre**
- Communication principalement en **Bluetooth Low Energy**

Interopérabilité

- Manque général d’interopérabilité — chaque wearable a son app + cloud propriétaire
- Norme **Continua** (santé connectée) — peu d’appareils compatibles

Réglementation (dispositif médical)

- Suisse : législation **ODim** depuis 2021
- Apps médicales = minimum **classe IIa** → évaluation par organisation tierce à chaque mise à jour
- Conséquence : la majorité des apps sont dans la catégorie **fitness** ou **bien-être** (et non médicale) pour éviter la procédure

Wear OS (montres connectées)

- Lancé en 2014 sous le nom **Android Wear**
- Évolutions :
 - Wear 1.0 (2014)** : smartphone obligatoire (pas de Wi-Fi/LTE)
 - Wear 2.0 (2017)** : Wi-Fi + LTE, apps autonomes
 - Wear OS 3.0 (2022)** : fusion avec Tizen, tuiles tierces, personnalisation constructeurs
 - 4.0 (2023), 5.0 (2024), 6.0 (2025) — optimisations

Concurrents :

- Apple Watch (iOS only)

- Samsung Tizen → Wear OS fin 2021
- Garmin, Fitbit
- Sauf Apple Watch, toutes appairables avec Android **ou** iOS

Communication montre ↔ smartphone

- Appairage **BLE** obligatoire
- Wear 1.0 : montre = proxy via BLE → smartphone
- Wear 2.0+ : montre peut accéder directement à Internet via Wi-Fi/LTE
 - Privilège toujours le smartphone si proche (économie d’énergie)
 - Wi-Fi/LTE utilisé : smartphone absent, montre en charge, ou téléchargement volumineux

Développement Wear OS

- Très similaire à une app Android (Activités, ViewModels, Jetpack — **Compose recommandé**)
- Fragments non recommandés**
- Effort particulier sur l’UI — **défilement vertical à privilégier**
- 4 surfaces de développement :
 - Notifications** — alertes
 - Complications** — petits éléments sur le cadran
 - Tuiles** — pages glissables, données rapides
 - Application** — fonctionnalités complètes
- Exemple météo : Complication (T° actuelle) / Notification (alertes) / Tuile (prévisions du jour) / App (semaine + paramètres)

Principe : ne pas réimplémenter toute l’app smartphone — fonctionnalités phares en **2-3 taps** avec UI épurée.

Wearable Data Layer API (Play Services)

3 mécanismes selon les besoins :

	Data Client	Message Client	Channel Client
> 100 kb	Yes	No	Yes
Réseau (cloud)	Yes (évtl BT)	BT only	BT only
Vers nœud déconnecté	Yes (sync diff.)	No	No
Device → device	No (via cloud)	Yes	Yes (bidir.)
Fan out	Yes	No	No
Fiabilité	Yes	Yes (si BT)	Yes

- Data Client** : espace de stockage privé synchronisé entre les nœuds
- Message Client** : appels RPC à payload limité
- Channel Client** : transfert de données (streaming audio/vidéo)

NDK (Native Development Kit)

Généralités

- Permet d’utiliser du C/C++ sur Android
- Cross-compilé en **fichiers .so** (dynamically linked shared libraries) pour chaque architecture
- N’est PAS une alternative au SDK Kotlin/Java** — usage réservé à :
 - Performance maximale : traitement image/vidéo, compression, moteur de jeu, réseaux de neurones
 - Réutilisation de librairies C/C++ existantes

Contenu & Installation

- Géré par **Android Studio** (SDK Manager → NDK Side by Side)
- Toolchain **LLVM** : **CLANG** (compilation) + **LLD** (linkage)
- STL** fournie par **LLVM’s libc++** (C++14 défaut, C++17, partiellement C++20)
- API Android : <android/log.h>, <android/sensor.h>, <android/asset_manager.h>, <android/permission_manager.h> (API 31)

Architectures supportées :

Name	arch	ABI	triple
32-bit ARMv7	arm	armeabi-v7a	arm-linux-androideabi
64-bit ARMv8	aarch64	aarch64-v8a	aarch64-linux-android
32-bit Intel	x86	x86	i686-linux-android
64-bit Intel	x86_64	x86_64	x86_64-linux-android

Approches d’intégration

- JNI (Java Native Interface)** : code Kotlin/Java appelle des méthodes natives
- native-activity** : activité entièrement en C/C++ avec OpenGL ES — principalement pour jeux vidéo
- makefile Android** ou **CMake** (recommandé, surtout cross-plateformes)

Intégration JNI — Exemple

Structure projet : dossier cpp/ dans src/main/ avec CMakeLists.txt et sources (.h, .c, .cpp).

Référencer le CMake dans build.gradle (app) :

```

configure<ApplicationExtension> {
    externalNativeBuild {
        cmake {
            path = file("src/main/cpp/CMakeLists.txt")
            version = "4.1.2"
        }
    }
}
```

CMakeLists.txt :

```

cmake_minimum_required(VERSION 3.31)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE} -O3")
add_library(native-lib SHARED native-lib.cpp)
find_library(log-lib log) // librairie log du NDK
target_link_libraries(native-lib ${log-lib})
```

Côté Kotlin (déclaration sans corps + chargement de la lib) :

```

class MainActivity : AppCompatActivity() {
    companion object {
        init { System.loadLibrary("native-lib") }
    }
    private external fun nativeSum(v1: Int, v2: Int): Int

    override fun onCreate(...) {
        val res = nativeSum(1, 2) // Δ éviter le UI-Thread
    }
}
```

Côté C++ (signature générée par l’IDE — convention

```

Java_package_<class>_<method>):
#include <jni.h>
extern "C"
JNIEXPORT jint JNICALL
Java_ch_heigvd_iict_dma_mynativeapplication_MainActivity_nativeSum(
    JNIEnv* env, jobject thiz, jint v1, jint v2) {
    return v1 + v2;
}
```

JNI — Types primitifs

Java	JNI	Bytes	Kotlin
boolean	jboolean	1 unsigned	Boolean
int	jint	4 signed	Int
long	jlong	8 signed	Long
short	jshort	2 signed	Short

byte	jbyte	1 signed	Byte
char	jchar	2 unsigned	Char
float	jfloat	4	Float
double	jdouble	8	Double
void	void	—	Unit

JNI — Types avancés

- Object ↔ jobject ↔ Any
- String ↔ jstring ↔ String (UTF-16 Java/Kotlin vs UTF-8 C++)
- Class ↔ jclass ↔ Class<>
- int[] ↔ jintArray ↔ IntArray (primitif, ≠ Array<Int> = Integer[])
- Hiéarchie : jobject → jclass, jstring, jarray (jobjectArray, jintArray, jdoubleArray...), throwable

Conversion jintArray ↔ std::vector<int> (seul le conteneur change, jint = int) :

```

inline std::vector<int> convert(JNIEnv* env, const jintArray &values) {
    jsize len = env->GetArrayLength(values);
    jint* content = env->GetIntArrayElements(values, nullptr);
    std::vector<int> v(content, content + len);
    env->ReleaseIntArrayElements(values, content, JNI_ABORT);
    return v;
}
jintArray convert(JNIEnv* env, const std::vector<int> &values) {
    jintArray arr = env->NewIntArray(values.size());
    env->SetIntArrayRegion(arr, 0, values.size(), (jint*) &values[0]);
    return arr;
}
```

Conversion jstring ↔ std::string (via réflexion sur getBytes("UTF-8")) — code complexe.

Exemple d’application 1 — tri natif

Tri d’un IntArray aléatoire (1M éléments) sur Samsung A32 :

- Kotlin .sorted() : 1467 ms
- Natif (std::sort) sans optim : 431 ms (+3.5)
- Natif avec -O3 : 99 ms (+14)

```

private suspend fun sortNative(n: Int) =
withContext(Dispatchers.Default) {
    val unsorted = randomValues(n)
    val time = measureNanoTime
    { values.postValue(nativeSort(unsorted).asList()) }
}
```

Exemple d’application 2 — analyse fertilité

3 modules natifs : pilote USB caméra, FFMPEG (compression vidéo), algorithmes OpenCV (analyse vidéo réutilisable Mac/Windows/iOS).

Optimisation

- Flag d’optimisation -Oz conseillé (**taille** + **vitesse**)
- ⚠ L’APK contient **4 copies** de toutes les librairies (1 par architecture) → APK volumineux
 - Exemple OpenCV + FFMPEG : 242 Mo de libs pour les 4 archis
- Solution : Android App Bundle (AAB)** → distribuer uniquement l’architecture cible du device

Instant App & App Clips

- Android** : Instant Apps depuis octobre 2017 — **terminé fin 2025** → remplacé par **Play Feature Delivery**
- Apple** : App Clips depuis iOS 14 (septembre 2020)
- Apps natives sans installation préalable
 - Téléchargement et ouverture auto (lien, QR, NFC)

- ↳ Désinstallation après usage
- Tournent dans une **sandbox** avec restrictions importantes

Cas d’usage : location vélo, paiement parking, commande restaurant, configuration appareil connecté → utilisation ponctuelle (**24% des apps ne sont ouvertes qu’une fois**, 39.9% désinstallées car non utilisées).

Instant App Android :

- Plusieurs points d’entrée par app (deep linking)
- Taille max **15 Mo**
- Possibilité de migration sandbox → app complète installée

Cross-plateformes

Le spectre web → natif

Approche	Position
Responsive web	web pur
Progressive Web App (PWA)	web installable
WebView (Cordova/Ionic)	hybride
VM (Flutter, React Native, MAUI)	proche natif
Code/lib commun (Kotlin MP)	natif

Évolution : **de plus en plus proche du natif** pour performances, UX, accès aux API mobiles.

Web App

- Navigateurs mobiles homogènes (HTML5/CSS/JS)
- Mais APIs avancées inégales selon le navigateur :

API	Chr. And	FF And	WebV And	Safari iOS	WebV iOS
Storage	✓	✓	✓	✓	✓
Bluetooth	✓	×	×	×	×
Geolocation	✓	✓	✓	✓	×
Sensor	✓	×	✓	×	×
NDEFReader	✓	×	×	×	×

Progressive Web App (PWA)

- Web app **installable** (raccourci launcher, fonctionne hors-ligne)
- Minimum requis :
 - Manifest** (json décrivant l’app)
 - Service Worker** (proxy pour cache des fichiers)
- Support iOS s’améliore mais pas total

Apache Cordova (ex PhoneGap)

- App hybride : **WebView** exécutant web app + librairies JS pour API natives via **plugins natifs**
- Base de **Ionic** (Angular, React, Vue)
- Successeur «spirituel» : **Capacitor** by Ionic

Solutions basées sur une VM

- Composants principaux :
- Machine virtuelle** exécutant code Dart/JS/C#
 - Moteur de rendu graphique**
 - Modules natifs** pour les API spécifiques plateforme

3 solutions principales :

Framework	Éditeur	Langage	Licence
Flutter	Google	Dart	BSD
React Native	Meta	JavaScript	MIT
.NET MAUI	Microsoft	C#	MIT (ex-Xamarin)

Note : **Unity** (C#) fait du cross-plateformes pour jeux.

React Native

- Mappe l’UI sur des **widgets natifs** (UIView, ViewGroup) → interaction fluide
- JS interprété par une **VM** spécifique à la plateforme
- Sous Android, VM = .so : libjsc.so (JavaScriptCore) — remplacé par **Hermes** en 2023
- Packages natifs Objective-C + Java pour API SDK (ex. Géolocalisation : interface JS commune)

OTA (Over The Air) : JS interprété → l’app peut télécharger une nouvelle version sans passer par le store. Apple ne l’interdit pas explicitement mais (2.4.5) interdit l’ajout de fonctionnalités majeures sans review.

Flutter

- SDK Google cross-plateformes (Android, iOS, Linux, macOS, Windows, Fuchsia, web, embarqué)
- Composants : **Dart** (JIT debug / AOT release), **Foundation Library**, **moteur Impeller** (C/C++), **Widgets** (Material + Cupertino), **Embedder** (couche plateforme : rendu, plugins, packaging)
- Architecture en couches : Framework Dart → Engine C/C++ → Embedder (platform-specific)
- Web : engine cible HTML/CSS/Canvas/WebGL/WebAssembly

Kotlin Multiplatform (JetBrain)

- Approche **à la carte** — granularité variable (module ↔ app complète)
- Couches partageables : **Data/Core**, **Business/Domain**, **Presentation**

3 niveaux de partage :

Minimal	Intermédiaire	Complet
Logique seule	Data + Business + Présentation	UI partagée via Compose MP

- Compose Multiplatform** : Android/iOS/Windows/macOS/Linux stables, Web bêta

Langage Dart

Caractéristiques

- Syntaxe «à la Java»
- Inférence de type** : int i = 2; ou var j = 2;
- Null-safety** : int i = null; ne compile pas; int? i = null; compile
- Pas de types primitifs** — tout est objet
- Test en ligne** : https://dartpad.dev/
- 3 modes d’exécution** :
 - Natif / AOT : exécutable natif (mode release)
 - Autonome : VM Dart (mode debug)
 - Transpilé en JavaScript (web ; WebAssembly aussi)

Paramètres de méthode

Positionnels par défaut (entre []) :

```
void test(String name, [String firstname = "", int age = 0]) {...}
test("Toto"); test("Toto", "Tata"); test("Toto", "Tata", 5);
```

Nommés par défaut (entre { }) — appel nominatif **obligatoire** :

```
void test(String name, {String firstname = "", int age = 0}) {...}
test("Toto", firstname: "Tata", age: 5);
test("Toto", "Tata", 5); // NE COMPILE PAS
```

Classes & Constructeur

```
class Person {
  late String name; // late = init plus tard
  late String firstname; // String non null ; String? peut être null
  Person(String firstname, String name) {
    this.name = name; this.firstname = firstname;
  }
  @override String toString() { return "$firstname $name"; }
}
var p = Person("Toto", "Tata"); // new optionnel, type inféré
```

Sucre syntaxique — **initialisation formelle** + **fonction fléchée** :

```
class Person {
  String name; String firstname;
  Person(this.firstname, this.name);
  @override String toString() => "$firstname $name";
}
```

Getters / Setters / Privé

- Variables préfixées par _ = **privées** (pas une convention — c’est le langage)
 - Pas de mot-clé private
- ```
String _firstname;
String get firstname => _firstname;
set firstname(String value) {_firstname = value; }
```

### Héritage

```
class Student extends Person {
 String school;
 Student(String firstname, String name, this.school) :
super(firstname, name);
 @override String toString() => "${super.toString()} - $school";
}
```

## Flutter — Première app

### Installation

- Téléchargement zip → ajouter flutter/bin au PATH
- flutter doctor : vérifie installation (Android SDK, Xcode, IDE, plugins...)
- flutter devices : liste les cibles disponibles
- Plugins recommandés : **Flutter** + **Dart** dans Android Studio / VS Code

### Structure du projet

```
my_first_app/
 android/ ios/ linux/ macos/ web/ windows/ + coquilles natives
 lib/main.dart + code Flutter
 test/
 pubspec.yaml + dépendances + assets
```

**Coquille Android** :

```
class MainActivity : FlutterActivity() {}
// FlutterActivity :
// - accueille l'UI Flutter en plein-écran
// - affiche launchscreen/splashscreen
// - charge et lance le code Dart (main() par défaut)
// - sauve et restaure l'état
```

### Point d’entrée

```
void main() { runApp(const MyApp()); }
```

```
class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return MaterialApp(
 title: 'Flutter Demo',
 theme: ThemeData(colorScheme: ColorScheme.fromSeed(seedColor:
Colors.indigo)),
 home: const MyHomePage(title: 'My First Flutter Home Page'),
);
 }
}
```

```
);
}
}
}
main() lance l’app (widget sans état)
build() représente l’UI graphique — appelée à l’affichage et à chaque «recomposition»
Tout est widget (même l’application)
```

**APK généré** : 17 Mo dont l’essentiel est natif

- classes.dex : MainActivity Kotlin + coquille
- libapp.so : code Dart compilé en natif
- libflutter.so : moteur de rendu Flutter

## Flutter — Widgets de base

### Layouts

- Single-child** : Center, Align, Padding, SizedBox, Transform, Container
- Multi-child** : Column, Row, Stack, GridView, ListView

| Layout   | Usage                                              |
|----------|----------------------------------------------------|
| Center   | centre un enfant                                   |
| Align    | Alignment.bottomRight, etc.                        |
| Padding  | EdgeInsets.only(top: 150, left: 250, ...)          |
| Column   | enfants empilés verticalement                      |
| Row      | enfants alignés horizontalement, mainAxisAlignment |
| SizedBox | espacement fixe ou container dimensionné           |
| Stack    | superposition                                      |

**Row avec espacement** :

```
Row(
 mainAxisAlignment: MainAxisAlignment.center,
 children: [
 TextButton(onPressed: (){}), child: Text("Btn 1")),
 SizedBox(width: 42),
 ElevatedButton(onPressed: (){}), child: Text("Btn 2")),
],
)
```

### ListView (statique vs dynamique)

**Statique** (children explicites) :

```
ListView(padding: EdgeInsets.all(8), children: [Container(...),
Container(...), ...])
```

**Dynamique avec builder** (recommandé pour grosses listes — lazy) :

```
ListView.builder(
 itemCount: items.length,
 itemBuilder: (context, index) {
 return Container(height: 50,
 child: Row(children: [Icon(Icons.adb), Text(items[index])]));
 },
);
```

### Boutons Material (3 types)

- TextButton** : sans fond ni bordure → actions secondaires
- ElevatedButton** : fond coloré + élévation → action principale
- OutlinedButton** : bordure sans fond → actions alternatives
- Variantes .icon: ElevatedButton.icon(icon: ..., label: ...)

### Images

- Widget Image supporte JPEG/PNG/GIF/WebP/animatedWebP/BMP/WBMP
- Paramètres : width, height, fit (BoxFit.cover/contain/fit/fitWidth...)
- Sources : asset locale ou réseau

**Ajout d'une image asset :**

1. Créer Assets/Images/ à la racine
2. Y déposer l'image (ex. android.jpg)
3. Référencer dans pubspec.yaml (format YAML, sensible à l'indentation) :

```
dependencies:
 flutter: { sdk: flutter }
 cupertino_icons: ^1.0.5 # ≡ >=1.0.5 <2.0.0
flutter:
 uses-material-design: true
 assets:
 - Assets/Images/android.jpg
```

1. Cliquer **Pub get** dans l'IDE

**Commandes Pub :**

- pub get : récupère et fixe les versions
- pub upgrade : met à jour sans tenir compte des versions fixées
- pub outdated : liste les dépendances obsolètes

**Utilisation :**

```
Image.asset("Assets/Images/android.jpg", width: 400)
Image.network("https://heig-vd.ch/logo.png", width: 250)
// permission INTERNET ajoutée auto en mode debug (pour hot-reload)
```

Flutter — Widgets à état

StatelessWidget vs StatefulWidget

- **Stateless** : UI immuable (build() une seule fois pour une config donnée)
- **Stateful** : UI dynamique avec état — déclencher setState() provoque un rebuild

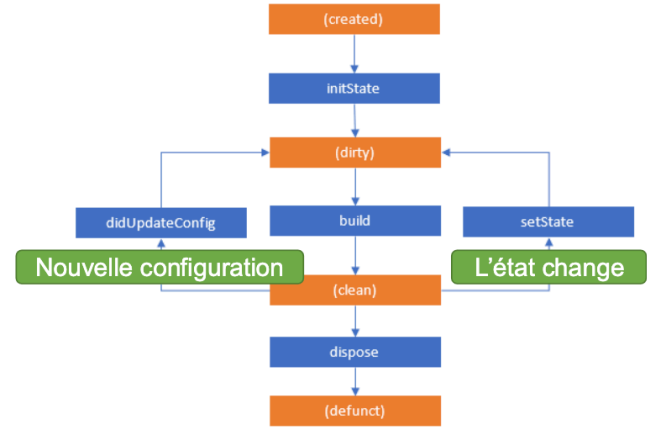
**Problème classique** — Slider dans Stateless : l'événement onChanged se déclenche mais le slider ne bouge pas (la valeur n'est jamais mise à jour visuellement).

**Solution Stateful** (2 classes) :

```
class TestPage extends StatefulWidget {
 const TestPage({Key? key}) : super(key: key);
 @override State<TestPage> createState() => _TestPageState();
}

class _TestPageState extends State<TestPage> {
 double _value = 25;
 void changeValue(double v) { setState((){ _value = v; }); }
 @override
 Widget build(BuildContext context) {
 return Center(child: Slider(
 value: _value, min: 0, max: 100, onChanged: changeValue,
));
 }
}
```

Cycle de vie d'un Stateful



- initState() : init ressources/streams/controllers (super en premier)
- build() : appelée après initState, chaque setState, didUpdateWidget — doit être pure
- didUpdateWidget() : parent rebuild avec nouvelle config
- setState() : marque dirty → rebuild à la prochaine frame
- dispose() : libérer streams/timers/controllers (sinon fuites)
- **État dans le Widget** = couplage logique/UI → préférer Stateless + state management externe

Compteur démo (généré par défaut)

```
class _MyHomePageState extends State<MyHomePage> {
 int _counter = 0;
 void _incrementCounter() { setState(() { _counter++; }); }
 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(title: Text(widget.title)),
 body: Center(child: Column(
 mainAxisAlignment: MainAxisAlignment.center,
 children: [
 const Text('You have pushed the button this many times:'),
 Text('$counter', style:
 Theme.of(context).textTheme.headlineMedium),
],
)),
 floatingActionButton: FloatingActionButton(
 onPressed: _incrementCounter,
 tooltip: 'Increment',
 child: const Icon(Icons.add),
),
);
 }
}
```

Flutter — Architecture

Approches de gestion d'état

- **Provider** — approche historique recommandée (toujours valide)
- **Riverpod** — recommandé aujourd'hui par le dev de Provider (anagramme)
- **BLoC** — alternative/complémentaire
- Autres : Redux/Fish-Redux, MobX, GetIt, GetX

Provider

- Librairie tierce (Flutter Favorite) — wrapper autour d'InheritedWidget
- Évite l'usage de Stateful

**Ajout au projet :**

```
dependencies:
 flutter: { sdk: flutter }
 provider: ^6.1.5
```

**Modèle (ChangeNotifier) :**

```
class ColorProvider extends ChangeNotifier {
 int _r, _g, _b; double _o;
 ColorProvider(this._r, this._g, this._b, this._o);
 get color => Color.fromRGB0(_r, _g, _b, _o);
 get r => _r; // getters individuels
 // ...
 void setColor({int? r, int? g, int? b, double? o}) {
 if(r != null) _r = r; /* ... */
 notifyListeners(); // Δ propage le changement
 }
}
```

**Racine de l'app — ChangeNotifierProvider :**

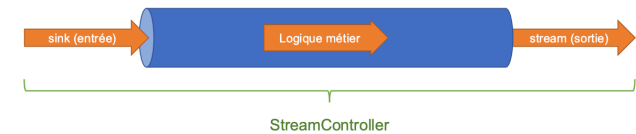
```
MaterialApp(
 home: ChangeNotifierProvider(
 create: (_) => ColorProvider(255, 255, 255, 1.0),
 child: const ColorPickerPage("Colorpicker"),
),
);
```

**Consommation dans un StatelessWidget :**

```
final colProv = Provider.of<ColorProvider>(context);
// colProv.color, colProv.r, ...
Slider(value: colProv.r.toDouble(), min: 0, max: 255,
 onChanged: (v) { colProv.setColor(r: v.round()); },
```

BLoC (Business Logic Component)

- Design pattern séparant UI / logique métier / data
- Repose sur les **Stream** natifs Dart (pas de lib externe), orchestré par un **StreamController**



**BLoC ColorPicker :**

```
class ColorBloc {
 static const defaultColor = Color.fromRGB0(255, 255, 255, 1.0);
 Color _color = defaultColor;
 final _streamController = StreamController<Color>();

 ColorBloc() { sink.add(_color); }

 Sink<Color> get sink => _streamController.sink;
 Stream<Color> get stream => _streamController.stream;

 void setColor({int? r, int? g, int? b, double? o}) {
 if(r != null) _color = _color.withRed(r);
 /* ... */
 sink.add(_color);
 }
 dispose() => _streamController.close();
}
```

**UI avec StreamBuilder** (Stateful uniquement pour le cycle de vie / dispose) :

```
class ColorPicker extends State<ColorPickerPage> {
 final ColorBloc _colorBloc = ColorBloc();
 @override
 Widget build(BuildContext context) {
 return StreamBuilder<Color>(
```

```
stream: _colorBloc.stream,
initialData: ColorBloc.defaultColor,
builder: (context, snapshot) {
 return Scaffold(* utilise snapshot.data */);
},
);
}
@override
void dispose() { super.dispose(); _colorBloc.dispose(); }
}
```

Provider vs BLoC

| Critère      | Provider              | BLoC                      |
|--------------|-----------------------|---------------------------|
| Complexité   | Simple                | Moyenne                   |
| Base         | ChangeNotifier        | Stream / StreamController |
| Widget UI    | Stateless             | Stateful (dispose)        |
| Notification | notifyListeners()     | sink.add(val)             |
| Consommation | Provider.of<T>(ctx)   | StreamBuilder<T>          |
| Nullable     | Non — getters directs | Oui — snapshot.data       |
| Idéal pour   | État UI, formulaires  | DB, API réseau, flux      |

Flutter — Pop-up & Navigation

SnackBar

```
final _snackBar = SnackBar(
 content: const Text("Contenu du SnackBar"),
 duration: const Duration(seconds: 4),
 action: SnackBarAction(label: "Clic", onPressed: () { /* ... */ }),
);
// Affichage
ScaffoldMessenger.of(context).showSnackBar(_snackBar);
```

Dialog (AlertDialog)

```
showDialog(
 context: context,
 builder: (BuildContext buildContext) => AlertDialog(
 title: const Text("Information"),
 content: const Text("Ceci est un message d'information"),
 actions: [
 TextButton(
 onPressed: () {
 Navigator.of(buildContext).pop(); // ferme le dialog
 },
 child: const Text("Fermer"),
),
],
),
);
```

⚠ Contrairement au SnackBar, le Dialog nécessite de gérer la **Navigation** pour se fermer.

Routes & Navigation

- **Navigator** gère la pile d'écrans (push/pop)
- push : ajoute un écran, pop : revient en arrière
- **Routes nommées** recommandées (forme URL)

Push direct (MaterialPageRoute) :

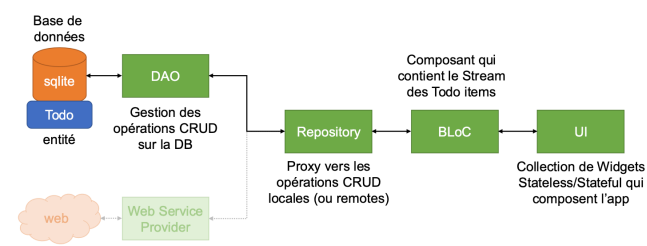
```
Navigator.push(context,
 MaterialPageRoute(builder: (context) => const PageTwo("Page
two")));
Navigator.pop(context);
```

Routes nommées (dans MaterialApp) :

```
MaterialApp(
 home: const PageOne("Page one"),
 routes: <String, WidgetBuilder> {
 '/p1': (context) => const PageOne("Page one"),
 '/p2': (context) => const PageTwo("Page two"),
 '/p3': (context) => const PageThree("Page three"),
 },
);
Navigator.pushNamed(context, "/p1");
```

Flutter — Exemple avancé Todo list

Architecture



Chaque couche ne connaît que la suivante → testable, remplaçable. DB stockée dans app\_flutter/ (dossier privé).

Librairies SQLite

- **sqflite** : plugin SQLite Flutter (iOS, Android, macOS — Linux/Windows via autre lib)
- **path\_provider** : trouver les espaces de stockage selon la plateforme
- **path** : manipuler les chemins multi-plateformes

Modèle entité Todo

```
class Todo {
 int? id;
 String description;
 bool isDone;
 Todo({this.id, this.description = "", this.isDone = false});
 // id = null avant insert, assigné par SQLite via AUTOINCREMENT

 factory Todo.fromMap(Map<String, dynamic> d) => Todo(
 id: d['id'], description: d['description'],
 isDone: d['is_done'] == 0 ? false : true, // int ⇄ bool
);
 Map<String, dynamic> toMap() => {
 "id": id, "description": description,
 "is_done": isDone == false ? 0 : 1,
 };
}
```

DatabaseProvider (singleton lazy)

```
class DatabaseProvider {
 static final DatabaseProvider dbProvider = DatabaseProvider();
 Database? _database;

 Future<Database> get database async {
 if (_database != null) return _database!;
 _database = await _createDatabase();
 return _database!;
 }
 Future<Database> _createDatabase() async {
 Directory docs = await getApplicationDocumentsDirectory();
 String path = join(docs.path, "todo.db");
 return await openDatabase(path, version: 1,
 onCreate: _initDB, onUpgrade: _onUpgrade);
 }
 void _initDB(Database db, int v) async {
```

```
 await db.execute("CREATE TABLE \$TODO_TABLE (
 "id INTEGER PRIMARY KEY, description TEXT, is_done INTEGER)");
 }
 void _onUpgrade(Database db, int old, int newV) {
 if (newV > old) { /* migration */ }
 }
}
```

SQLite n'a pas de format boolean → on utilise INTEGER (0/1).

DAO (CRUD)

```
class TodoDao {
 final dbProvider = DatabaseProvider.dbProvider;

 Future<int> createTodo(Todo t) async {
 final db = await dbProvider.database;
 return db.insert(TODO_TABLE, t.toMap());
 }
 Future<int> updateTodo(Todo t) async {
 final db = await dbProvider.database;
 return await db.update(TODO_TABLE, t.toMap(),
 where: "id = ?", whereArgs: [t.id]);
 }
 Future<int> deleteTodo(int id) async {
 final db = await dbProvider.database;
 return await db.delete(TODO_TABLE, where: 'id = ?', whereArgs:
[id]);
 }
 Future<List<Todo>> getTodos({String? query}) async {
 final db = await dbProvider.database;
 var result = (query != null && query.isNotEmpty)
 ? await db.query(TODO_TABLE,
 where: 'description LIKE ?', whereArgs: ["%$query%"])
 : await db.query(TODO_TABLE);
 return result.map((item) => Todo.fromMap(item)).toList();
 }
}
```

Repository (pont vers DAO)

```
class TodoRepository {
 final todoDao = TodoDao();
 Future getAllTodos({String? query}) => todoDao.getTodos(query:
query);
 Future insertTodo(Todo t) => todoDao.createTodo(t);
 Future updateTodo(Todo t) => todoDao.updateTodo(t);
 Future deleteTodoById(int id) => todoDao.deleteTodo(id);
}
```

BLoC

```
class TodoBloc {
 final _todoRepository = TodoRepository();
 final _todoController = StreamController<List<Todo>>();
 get todos => _todoController.stream;

 TodoBloc() { refreshTodos(); }

 refreshTodos({String? query}) async {
 _todoController.sink.add(await _todoRepository.getAllTodos(query:
query));
 }
 addTodo(Todo t) async {
 await _todoRepository.insertTodo(t);
 refreshTodos(); // refresh après CRUD
 }
}
```

UI avec StreamBuilder

```
StreamBuilder(
 stream: todoBloc.todos,
 builder: (context, AsyncSnapshot<List<Todo>> snapshot) {
 if (snapshot.hasData) return ListView.builder(* ... */);
 else return Center(child: loadingData());
 },
);
```

## Flutter — Internationalisation (i18n)

- Plugins : flutter\_localizations + intl, generate: true dans pubspec.yaml  
→ Flutter génère AppLocalizations
- Fichiers .arb (Application Resource Bundle, format JSON)
- 4 délégués nécessaires : AppLocalizations, GlobalMaterial, GlobalWidgets (LTR/RTL), GlobalCupertino

**MaterialApp :**

```
MaterialApp(
 localizationsDelegates: const [
 AppLocalizations.delegate,
 GlobalMaterialLocalizations.delegate,
 GlobalWidgetsLocalizations.delegate,
 GlobalCupertinoLocalizations.delegate,
],
 supportedLocales: const [
 Locale('en', ''), Locale('fr', ''),
],
);
```

**l10n.yaml** (racine projet) :

```
arb-dir: lib/l10n
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
```

**lib/l10n/app\_en.arb :**

```
{
 "todo": "To do",
 "@todo": { "description": "Title of the app" },
 "add_todo": "Start adding Todo...",
 "@add_todo": { "description": "Welcome message" }
}
```

**lib/l10n/app\_fr.arb :**

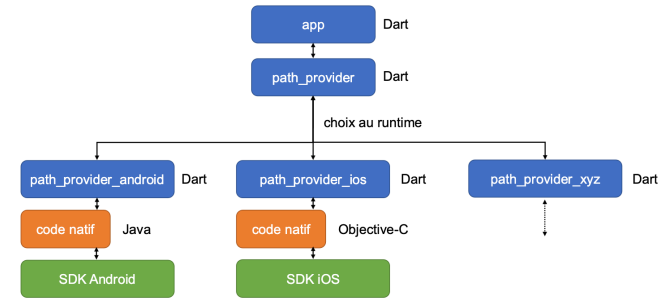
```
{ "todo": "À faire", "add_todo": "Veuillez créer une tâche pour commencer" }
```

**Utilisation :**

```
Text(AppLocalizations.of(context)!.add_todo)
```

## Flutter — Plugins multi-plateformes

### Architecture (exemple path\_provider)



Chaque plugin Flutter contient :

- **API Dart commune** exposée à l'app
- Implémentations **spécifiques par plateforme** (sous-packages distincts)
- Communication Dart ↔ natif via **MethodChannel** (canal bidirectionnel nommé, messages sérialisés) — transparent pour le dev

**Exemple** — `getApplicationDocumentsDirectory()` :

- Côté Dart : `_platform.getApplicationDocumentsPath()` (résolu au runtime)
- Côté Android (Kotlin/Java) : appelle `context.getDataDir()` (API 24+) ou fallback ; le plugin crée un dossier `app_flutter/`
- Côté iOS (Objective-C) : `NSSearchPathForDirectoriesInDomains(NSDocumentDirectory, ...)`

**Résultat** sur Android : DB stockée dans `/data/data/<pkg>/app_flutter/todo.db`

**Compatibilité path\_provider :**

| Dossier          | And | iOS | Lin | mac | Win |
|------------------|-----|-----|-----|-----|-----|
| Temporary        | ✓   | ✓   | ✓   | ✓   | ✓   |
| App Support      | ✓   | ✓   | ✓   | ✓   | ✓   |
| App Library      | ✗   | ✓   | ✗   | ✓   | ✗   |
| App Documents    | ✓   | ✓   | ✓   | ✓   | ✓   |
| External Storage | ✓   | ✗   | ✗   | ✗   | ✗   |
| Downloads        | ✗   | ✗   | ✓   | ✓   | ✓   |

Cross-plateforme : **Temporary** + **App Documents** sont universels.

## Récap Flutter

### Concepts clés

| Concept                   | Rôle                                              |
|---------------------------|---------------------------------------------------|
| Widget                    | Brique UI (tout est widget)                       |
| Stateless                 | UI immuable                                       |
| Stateful + setState()     | UI dynamique avec état interne                    |
| Provider / ChangeNotifier | State management, propagation via notifyListeners |
| BLoC + StreamController   | Logique métier réactive (sink/stream)             |
| StreamBuilder             | Reconstruit l'UI quand le Stream émet             |
| Navigator (push/pop)      | Pile d'écrans                                     |
| pubspec.yaml              | Dépendances + assets                              |
| .arb files + intl         | Internationalisation                              |

### Bonnes pratiques

- Préférer **Stateless** + **Provider/BLoC** aux Stateful purs (découpler UI/logique)
- **ListView.builder** pour grandes listes (lazy)
- **Dispose** obligatoire des StreamController dans le Stateful (`dispose()`)
- Éviter le hardcoding de strings → fichiers .arb
- Toujours `pub get` après modification de `pubspec.yaml`

### Récap NDK

- Usage = performance ou réutilisation de libs C/C++ existantes
- JNI : signatures C++ générées par l'IDE (`Java_<pkg>_<cls>_<method>`)
- `System.loadLibrary("native-lib")` dans le companion object Kotlin
- Compilation pour 4 archis → APK volumineux (-0z recommandé)
- Conversion `jXXXArray` ↔ `std::vector` : seul le conteneur change

### Récap Capteurs/Wearables

- `SensorManager` unifié, `registerListener/unregisterListener` (⚠️ onPause)
- `event.values` taille variable selon capteur
- Pool d'instances `SensorEvent` → ne pas conserver de référence
- Fusion multi-capteurs nécessaire pour reconnaissance d'activité fiable
- Wearable Data Layer API : `Data/Message/Channel` selon volume et besoins